

BayesForge Multi-Language Conversion Guide

This guide provides a reference for converting BayesForge (BF) code between Python, R (BFR), and Julia (BFJ).

Language Comparison Table

Feature	BF Python	BF R (BFR)	BF Julia (BFJ)
Import	<code>from BayesForge</code> <code>import bf</code>	<code>m = importBF()</code>	<code>m = importBF()</code>
Initialize	<code>m =</code> <code>bf(platform='cpu')</code>	<code>m =</code> <code>importBF(platform='cpu')</code>	<code>m =</code> <code>importBF(platform="cpu")</code>
Model	<code>def model(var1,</code> <code>var2):</code>	<code>model <-</code> <code>function(var1,</code> <code>var2) {</code>	<code>@BF function</code> <code>model(var1, var2)</code>
Priors	<code>m.dist.normal(0,</code> <code>1, name='x')</code>	<code>bf.dist.normal(0,</code> <code>1, name='x')</code>	<code>m.dist.normal(0, 1,</code> <code>name="x")</code>
Likelihood	<code>m.dist.normal(mu,</code> <code>sigma, obs=y)</code>	<code>bf.dist.normal(mu,</code> <code>sigma, obs=y)</code>	<code>m.dist.normal(mu,</code> <code>sigma, obs=y)</code>
Link (Logit)	<code>jax.nn.sigmoid(x)</code>	<code>jax\$nn\$sigmoid(x)</code>	<code>m.link.inv_logit(x)</code>
Exponential	<code>jnp.exp(x)</code>	<code>jnp\$exp(x)</code>	<code>jnp.exp(x)</code>
Logarithm	<code>jnp.log(x)</code>	<code>jnp\$log(x)</code>	<code>jnp.log(x)</code>
Indices	<code>x[idx]</code>	<code>x[idx]</code>	<code>x[idx]</code>
Fitting	<code>m.fit(model)</code>	<code>m\$fit(model)</code>	<code>m.fit(model)</code>
Summary	<code>m.summary()</code>	<code>m\$summary()</code>	<code>m.summary()</code>

Examples

1. Linear Regression

[BF Python]

```
def model(x, y):
    alpha = m.dist.normal(0, 10, name='alpha')
    beta = m.dist.normal(0, 1, name='beta')
    sigma = m.dist.exponential(1, name='sigma')
    mu = alpha + beta * x
    m.dist.normal(mu, sigma, obs=y)
```

[BF R (BFR)]

```
model <- function(x, y) {  
  alpha = bf.dist.normal(0, 10, name='alpha')  
  beta = bf.dist.normal(0, 1, name='beta')  
  sigma = bf.dist.exponential(1, name='sigma')  
  mu = alpha + beta * x  
  bf.dist.normal(mu, sigma, obs=y)  
}
```

[BF Julia (BFJ)]

```
@BF function model(x, y)  
  alpha = m.dist.normal(0, 10, name="alpha")  
  beta = m.dist.normal(0, 1, name="beta")  
  sigma = m.dist.exponential(1, name="sigma")  
  mu = alpha + beta * x  
  m.dist.normal(mu, sigma, obs=y)  
end
```

Tips for Conversion

1. Object Scope:

- In **Python**, `m` is the central object.
- In **R**, `bf.dist.*` is used for distributions globally once BF is imported, while `m` is used for methods like `fit` and `summary`.
- In **Julia**, `m` is used for both distributions (`m.dist`) and methods.

2. Accessing Modules:

- R uses `$` to access Python module members (e.g., `jax$nn$sigmoid`).
- Python and Julia use `.` (e.g., `jax.nn.sigmoid`).

3. Indexing:

Note that while R and Julia usually use 1-based indexing, the underlying JAX arrays in BF often follow the data indexing passed to the model. Best practice is to ensure your indices start from 0 if they are used to index JAX arrays.

4. R Integer Vectors:

In BF R, the `shape` parameter for distributions must be specified as an integer vector using the `L` suffix, for example: `shape = c(1L, 10L)`.

5. Macros:

Julia uses the `@BF` macro to correctly wrap the model function for the Python backend.